Microsoft Stock Price prediction using the LSTM(Long short-term memory) neural network.

In this project, we employed the LSTM neural network to predict Microsoft stock prices. LSTM, which stands for long short-term memory, is an advanced variant of the traditional recurrent neural network (RNN).

LSTM models are particularly adept at learning long-term dependencies in sequence prediction problems. They excel in recognizing patterns within data sequences, such as sensor data, stock prices, or natural language, by incorporating both the actual value and its position in the sequence during prediction.

What sets LSTM models apart is their unique architecture, enabling them to determine whether to retain or discard previous information in short-term memory. This capability allows them to effectively capture even longer dependencies within sequences.

By leveraging the power of LSTM networks, we aimed to predict Microsoft stock prices by uncovering patterns and dependencies in the historical data. This approach enables us to make informed forecasts and gain insights into potential future price movements.

Dataset

This project focuses on the exploration of the Microsoft stock data, which offers a wealth of information to unravel the intricacies of the stock market. The dataset comprises key variables that will guide our analysis.

The "Date" column serves as our timeline, enabling us to observe trends and patterns over time. The "Open Price" variable signifies the initial trading price, setting the stage for daily gains or losses. "High Price" and "Low Price" capture the price range, revealing market volatility and investor sentiment.

The "Closing Price" column denotes the day's final stock price, encapsulating market sentiment and the cumulative impact of various market forces. The "Adjusted Closing Price" considers factors like stock splits and dividends, providing an accurate representation of the stock's performance.

The "Volume" column reflects the number of shares traded, shedding light on market liquidity and investor interest.

Through this dataset, we aim to analyze trends, assess risk, and explore the interplay of market variables, ultimately uncovering valuable insights to make informed decisions.

Import Libraries

In [1]:
```
!pip install tensorflow-gpu==2.8.0
import tensorflow as tf

!pip install tensorflow-io==0.25.0

import numpy as np
import pandas as pd
import pandas_datareader as web
import math
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, LSTM
plt.style.use('fivethirtyeight')

import warnings

warnings.filterwarnings('ignore')
```

```
Requirement already satisfied: typing-extensions>=3.6.6 in /opt/conda/l
ib/python3.10/site-packages (from tensorflow-gpu==2.8.0) (4.5.0)
Requirement already satisfied: wrapt>=1.11.0 in /opt/conda/lib/python3.
10/site-packages (from tensorflow-gpu==2.8.0) (1.14.1)
Collecting tensorboard<2.9,>=2.8 (from tensorflow-gpu==2.8.0)
  Downloading tensorboard-2.8.0-py3-none-any.whl (5.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.8/5.8 MB 72.6 MB/s eta
0:00:00
Collecting tf-estimator-nightly==2.8.0.dev2021122109 (from tensorflow-g
pu==2.8.0)
  Downloading tf_estimator_nightly-2.8.0.dev2021122109-py2.py3-none-an
y.whl (462 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 462.5/462.5 kB 32.2 MB/s et
a 0:00:00
Collecting keras<2.9,>=2.8.0rc0 (from tensorflow-gpu==2.8.0)
  Downloading keras-2.8.0-py2.py3-none-any.whl (1.4 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 57.9 MB/s eta
0:00:00
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in
/opt/conda/lib/python3.10/site-packages (from tensorflow-gpu==2.8.0)
```

Import Dataset

In [2]:
```
df = pd.read_csv("/kaggle/input/microsoft-stock-data/MSFT.csv")

df
```

Out[2]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	1986-03-13	0.088542	0.101563	0.088542	0.097222	0.061434	1031788800
1	1986-03-14	0.097222	0.102431	0.097222	0.100694	0.063628	308160000
2	1986-03-17	0.100694	0.103299	0.100694	0.102431	0.064725	133171200
3	1986-03-18	0.102431	0.103299	0.098958	0.099826	0.063079	67766400
4	1986-03-19	0.099826	0.100694	0.097222	0.098090	0.061982	47894400
...	...	...	...	...	...	...	...
9078	2022-03-18	295.369995	301.000000	292.730011	300.429993	300.429993	43317000
9079	2022-03-21	298.890015	300.140015	294.899994	299.160004	299.160004	28351200
9080	2022-03-22	299.799988	305.000000	298.769989	304.059998	304.059998	27599700
9081	2022-03-23	300.510010	303.230011	297.720001	299.489990	299.489990	25715400
9082	2022-03-24	299.140015	304.200012	298.320007	304.100006	304.100006	24446900

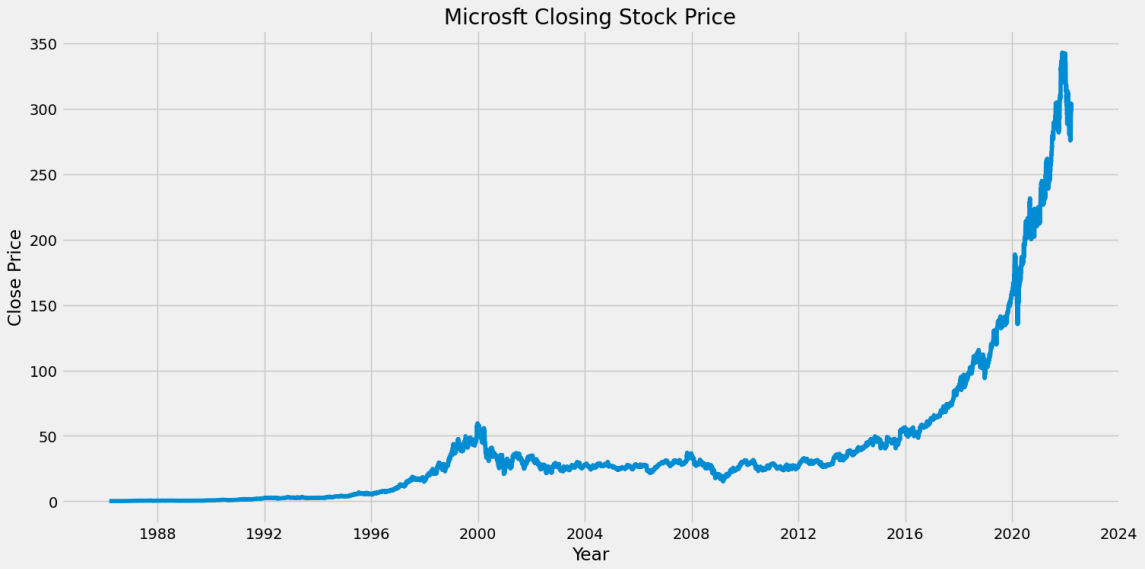
9083 rows × 7 columns

In [3]:
```
#convert date to datetime
df['Date']= pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 9083 entries, 1986-03-13 to 2022-03-24
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Open       9083 non-null   float64
 1   High       9083 non-null   float64
 2   Low        9083 non-null   float64
 3   Close      9083 non-null   float64
 4   Adj Close  9083 non-null   float64
 5   Volume     9083 non-null   int64  
dtypes: float64(5), int64(1)
memory usage: 496.7 KB
```

Basic overview of the stock price performance.

In [5]:
```
#visualize the closing prices to get a general idea about the stock price.
plt.figure(figsize=(16,8))
plt.plot(df['Close'])
plt.xlabel('Year')
plt.ylabel('Close Price')
plt.title('Microsft Closing Stock Price')
plt.show()
```



By examining this chart, we can observe the remarkable growth of Microsoft stock, starting from under a dollar and reaching a trading price of approximately $300.

In [6]:
```
#create dataframe for data and closing price
data1=df['Close']
```

Create and Scale Train data

In [7]:
```
#create new database with only required coloumns
data=df.filter(['Close'])
#convert the new dataframe to numpy array
dataset=data.values
#Use 80 percent of the number of rows to train
training_data_len=math.ceil(len(dataset)*0.8)
# get the number of rows for the 80% training data.
training_data_len
```

Out[7]: 7267

When working with neural networks, it is crucial to scale the data for optimal performance. Failing to scale the data can lead to subpar results. To address this, we will apply a standard scaler to the training data. First, we create an instance of the standard scaler and use it to scale the data. By employing the fit and transform functions on the training data, we obtain the scaled data. This scaling process ensures that the mean and standard deviation of the data are normalized to the range of 0 to 1, which aids in the neural network's effectiveness.

In [8]:
```
#scale the data
scaler=StandardScaler()
scaled_data=scaler.fit_transform(dataset)
print("MEAN of processed data: ",scaled_data.mean())
print("Standard deviation of processed data: ",scaled_data.std())
```

```
MEAN of processed data:  1.0013153162753806e-16
Standard deviation of processed data:  1.0
```

In [9]:
```
#create a new training data from the scaled training dataset
train_data = scaled_data[0:training_data_len, :]
#split the data to x_train and y_train
x_train=[]
y_train=[]
for i in range(60,len(train_data)):
  x_train.append(train_data[i-60:i])
  y_train.append(train_data[i])
```

In [10]:
```
#convert x_train and y_train into numpy arrays
x_train,y_train=np.array(x_train),np.array(y_train)
x_train.shape
```

Out[10]:
```
(7207, 60, 1)
```

In [11]:
```
#reshape the data
print("x_train shape before reshaping",x_train.shape)
x_train = np.reshape(x_train,(x_train.shape[0],x_train.shape[1],1)) #np.res
print("x_train shape after reshaping",x_train.shape)
```

```
x_train shape before reshaping (7207, 60, 1)
x_train shape after reshaping (7207, 60, 1)
```

In our project, we will be utilizing the sequential model for our neural network. Let's start by adding the first LSTM layer, which will consist of 200 neurons. The input shape for this layer will correspond to the number of input neurons in X train, ensuring compatibility.

Following that, we will add another LSTM layer, also with 200 neurons, resulting in a total of two LSTM layers. This layer configuration will help capture complex patterns and dependencies in the data.

Moving on, we will incorporate two dense layers into our model. The first dense layer will comprise 100 neurons, allowing for further feature extraction and representation. The second dense layer will consist of 50 neurons, enhancing the model's capacity to learn intricate relationships.

Lastly, we will add a dense layer of 1, serving as the output layer. This layer will produce a single output, enabling us to make predictions or classifications based on the network's learned representations.

By structuring our neural network in this manner, we aim to leverage the power of LSTM layers and dense layers to effectively capture and model the underlying patterns within our data.

Build the Model

In [12]:
```
#build LSTM model
model= Sequential()
model.add(LSTM(200,return_sequences=True,input_shape=(x_train.shape[1], 1))
model.add(LSTM(200,return_sequences=False))
model.add(Dense(100))
model.add(Dense(50))
model.add(Dense(1))
```

To proceed, we will compile the model and optimize it using the Adam optimizer, which is well-suited for a variety of deep learning tasks. Given that we are dealing with a regression problem, we will employ the mean squared error (MSE) as our loss metric. This choice aligns with our goal of minimizing the average squared difference between the predicted and actual values.

By utilizing the Adam optimizer and MSE loss metric, we can effectively train our model to make accurate predictions for our regression problem. This combination of optimization and loss function selection ensures that our model is optimized to minimize the error between predicted and actual values, ultimately enhancing its performance and ability to tackle regression tasks.

In [13]:
```
#compile the model
model.compile(optimizer='adam',loss='mean_squared_error')
```

In [14]:
```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 60, 200)           161600

 lstm_1 (LSTM)               (None, 200)               320800

 dense (Dense)               (None, 100)               20100

 dense_1 (Dense)             (None, 50)                5050

 dense_2 (Dense)             (None, 1)                 51

=================================================================
Total params: 507,601
Trainable params: 507,601
Non-trainable params: 0
_________________________________________________________________
```

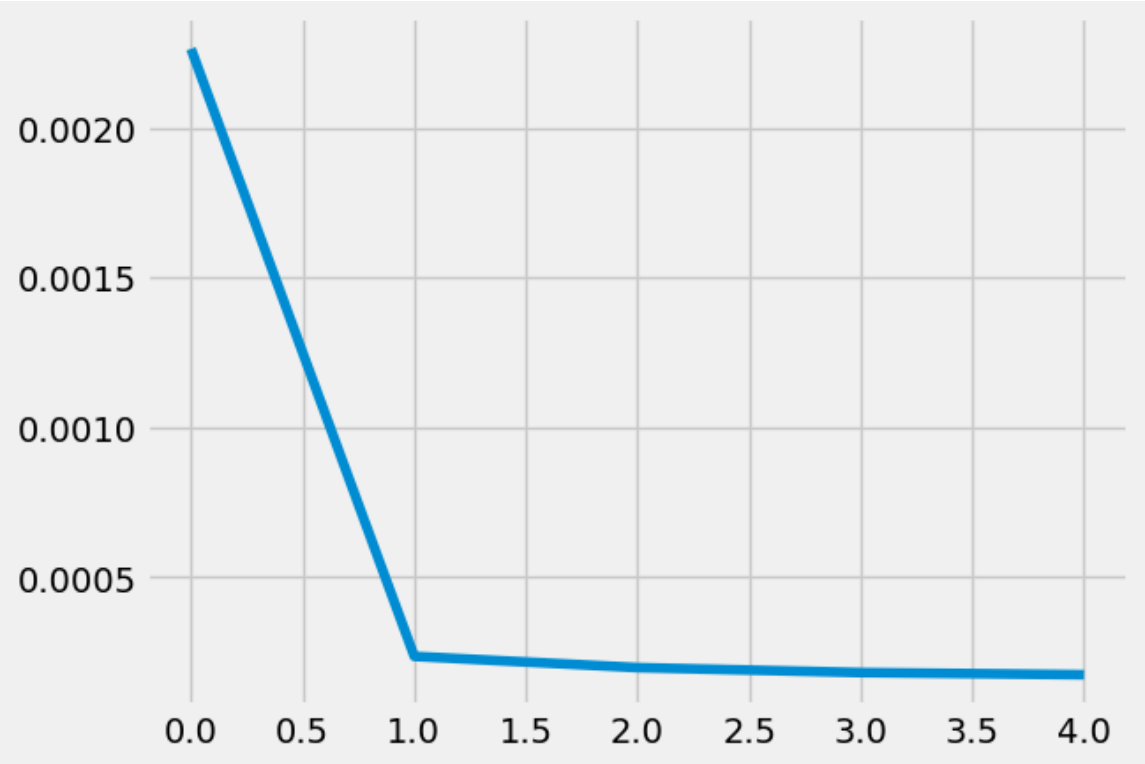
The model summary reveals that there are 507,601 trainable parameters, indicating the adjustable elements for learning. However, there are no non-trainable parameters present in the model, meaning all parameters can be updated during the training process.

In [15]:
```
#train the model. this model with five epochs. You can increase the number
history = model.fit(x_train,y_train,epochs=5)
```

```
Epoch 1/5
226/226 [==============================] - 46s 187ms/step - loss: 0.0023
Epoch 2/5
226/226 [==============================] - 43s 188ms/step - loss: 2.3589e-
04
Epoch 3/5
226/226 [==============================] - 43s 190ms/step - loss: 1.9841e-
04
Epoch 4/5
226/226 [==============================] - 43s 188ms/step - loss: 1.8189e-
04
Epoch 5/5
226/226 [==============================] - 43s 189ms/step - loss: 1.7466e-
04
```

In [16]:
```
#plot the history of loss.
plt.plot(history.history['loss'])
```

Out[16]:
```
[<matplotlib.lines.Line2D at 0x7b4e6e8d6bc0>]
```



The loss on the chart consistently decreased with each epoch, indicating a steady improvement in the model's performance over time.

Create Test data from Scaled Train data

In [17]:
```
#create the testing dataset
#create new array.
# test data is equal to scale data of training data length -60 including al
test_data=scaled_data[training_data_len-60:, :]
#create the dataset x_test and y_test
x_test=[]
y_test=dataset[training_data_len: , :]
for i in range(60,len(test_data)):
  x_test.append(test_data[i-60:i, 0])
```

In [18]:
```
#convert the data to numpy
x_test=np.array(x_test)
```

In [19]:
```
#reshape the data
x_test=np.reshape(x_test,(x_test.shape[0],x_test.shape[1],1))
```

Model Predictions

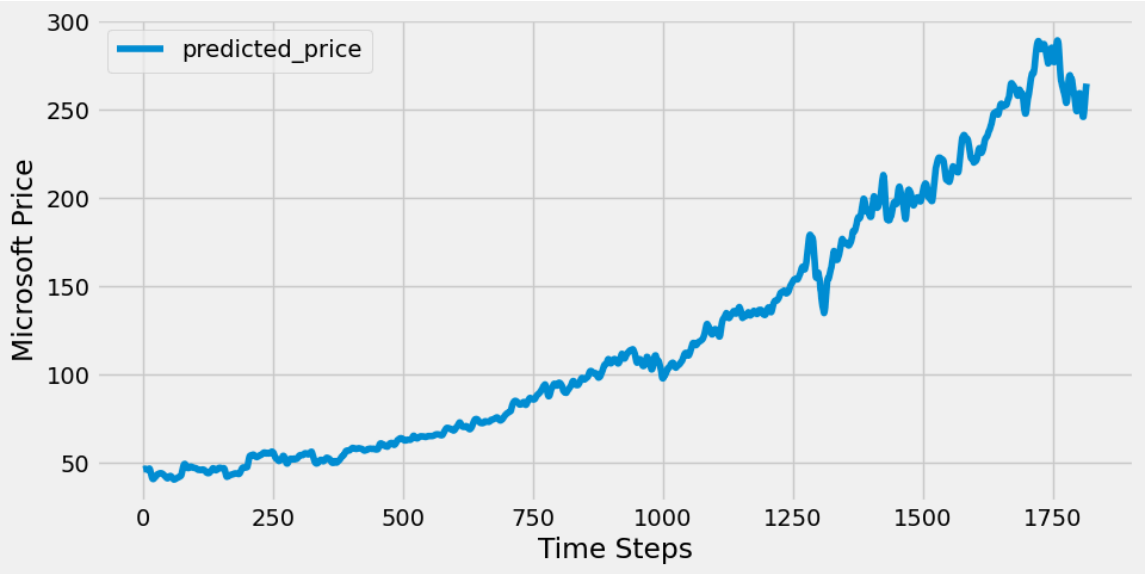
In [20]:
```
#get the models predicted value.
# check the predictions on X test
predictions =model.predict(x_test)
predictions=scaler.inverse_transform(predictions)
print(predictions)
```

```
[[ 46.334545]
 [ 46.400192]
 [ 46.584484]
 ...
 [259.022   ]
 [262.61783 ]
 [264.88297 ]]
```

These predictions represent the Microsoft stock prices generated by the LSTM model using the X test data.

In [21]:
```
predicted_price = pd.DataFrame(predictions, columns = ['price'] )
predicted_price.head()

#Plot the predicted price of microsoft stock
predicted_price.plot(figsize=(10,5))
plt.legend(['predicted_price'])
plt.ylabel("Microsoft Price")
plt.xlabel("Time Steps")
plt.show()
```



Analyzing the chart, we observe that the predicted price of Microsoft stock initiated at $45 and is projected to gradually ascend, surpassing 250 at its peak before experiencing a decline. This prediction suggests a positive trend in the stock's value over the given time period. It indicates potential growth opportunities for investors, reaching a substantial price level before a subsequent downturn. The chart provides a visual representation of the predicted price trajectory, aiding in decision-making and assessing investment strategies related to Microsoft stock.

In [ ]: